HASHCODE ACADEMY / MASTERCLASS

JavaScript & TypeScript

Comprendre l'écosystème. Lire le code. Construire un premier outil.

HASHCODE MASTERCLASS | CODE EN DIRECT | ATELIER GUIDÉ

LEARN · BUILD · BELONG

OUVERTURE / APPRENDRE PAR LA PRATIQUE

Le code s’apprend en faisant.

Observer → prédire → exécuter → expliquer → améliorer.

- On ne cherche pas à tout mémoriser.
- On apprend à raisonner sur le comportement du programme.
- Chaque concept sera relié à un exemple concret.

OUVERTURE / APPRENDRE PAR LA PRATIQUE

Le contrat de cette masterclass

Deux heures pour comprendre les fondamentaux, pas pour devenir expert de tout l'écosystème.

01

COMPRENDRE

Situer JavaScript, TypeScript, Node.js et npm.

02

PRATIQUER

Lire, écrire et corriger de petits programmes.

03

CONSTRUIRE

Assembler les concepts dans un gestionnaire de tâches.

OUVERTURE / APPRENDRE PAR LA PRATIQUE

Les résultats attendus

À la fin, vous devriez pouvoir…

- Expliquer où JavaScript s'exécute et à quoi sert Node.js.
- Manipuler variables, fonctions, tableaux et objets.
- Lire des annotations TypeScript et une interface.
- Expliquer ce que le typage détecte — et ce qu'il ne détecte pas.
- Exécuter et améliorer un mini-projet guidé.

OUVERTURE / APPRENDRE PAR LA PRATIQUE

Diagnostic express

Choisissez mentalement votre point de départ.

- A — Je découvre la programmation.
- B — Je connais variables, conditions ou fonctions.
- C — J'ai déjà réalisé un petit projet.

À VOUS

Question au groupe : qu'est-ce qui vous paraît le plus confus entre JavaScript, TypeScript et les frameworks ?

OUVERTURE / APPRENDRE PAR LA PRATIQUE

Notre itinéraire en 120 minutes

Des concepts courts, une démonstration, puis de la pratique.

01

00–15

Repères et écosystème

02

15–40

Exécution et outils

03

40–65

Fondamentaux JavaScript

04

65–82

Pourquoi TypeScript

05

82–108

Atelier Task Manager

06

108–120

Quiz, bilan et suite

CARTE DE L'ÉCOSYSTÈME / APPRENDRE PAR LA PRATIQUE

Un langage n'est pas un framework

Les outils JavaScript répondent à des problèmes différents.

- JavaScript : langage de programmation.
- TypeScript : JavaScript avec un système de types statiques.
- Node.js : environnement d'exécution JavaScript hors navigateur.
- npm : gestion des paquets et des scripts de projet.
- React : bibliothèque d'interface ; Next.js : framework basé sur React.

CARTE DE L'ÉCOSYSTÈME / APPRENDRE PAR LA PRATIQUE

La carte mentale en une ligne

Du code source jusqu'à l'application utilisée.

01

LANGAGE

JavaScript / TypeScript

02

EXÉCUTION

Navigateur / Node.js

03

OUTILS

npm / compilateur / Vite

04

PRODUIT

Site web / API / application

DE L'IDÉE AU RÉSULTAT 01 — 04

Écrire code source → **Exécuter** runtime → **Observer** résultat → **Corriger** itération

Le développeur alterne entre hypothèse, exécution et vérification. Une erreur est une information exploitable, pas un verdict.

CARTE DE L'ÉCOSYSTÈME / APPRENDRE PAR LA PRATIQUE

JavaScript et TypeScript

Ils ne sont pas deux langages concurrents au même niveau.

A

JavaScript

- Le code exécuté par les moteurs JavaScript.
- Types déterminés à l'exécution.
- Souple et directement utilisable dans le navigateur.

B

TypeScript

- Ajoute des annotations et des contrôles de types.
- Le code est transformé/émis pour l'environnement cible.
- Aide à repérer certaines erreurs avant l'exécution.

CARTE DE L'ÉCOSYSTÈME / APPRENDRE PAR LA PRATIQUE

Que fait le navigateur ?

Il fournit un environnement pour exécuter JavaScript côté client.

- Affiche le HTML et applique le CSS.
- Expose des API comme DOM, événements et fetch.
- Exécute le JavaScript avec son moteur.
- Applique des règles de sécurité, notamment la politique de même origine.

CARTE DE L'ÉCOSYSTÈME / APPRENDRE PAR LA PRATIQUE

JavaScript dans le navigateur

Un événement déclenche une fonction quand l'utilisateur clique.

```
const button = document.querySelector("button");

button?.addEventListener("click", () => {
  console.log("Action déclenchée !");
});
```

RÉSULTAT ATTENDU

```
Au clic : Action déclenchée !
```

CARTE DE L'ÉCOSYSTÈME / APPRENDRE PAR LA PRATIQUE

Que fait Node.js ?

Node.js permet d'exécuter JavaScript hors du navigateur.

- Lancer des scripts depuis un terminal.
- Créer des API et des services serveur.
- Lire des fichiers et automatiser des tâches.
- Utiliser l'écosystème npm pour installer des outils.

À RETENIR

Node.js est un runtime, pas un framework front-end.

CARTE DE L'ÉCOSYSTÈME / APPRENDRE PAR LA PRATIQUE

Navigateur ou Node.js ?

Le langage est proche ; les capacités de l'environnement changent.

A

Navigateur

- document / DOM
- Événements d'interface
- Stockage et API web

B

Node.js

- process / système de fichiers
- Serveurs et scripts
- Modules et API Node

CARTE DE L'ÉCOSYSTÈME / APPRENDRE PAR LA PRATIQUE

À quoi sert npm ?

npm aide à partager, installer et exécuter des outils JavaScript.

- Installer des dépendances réutilisables.
- Déclarer des scripts dans package.json.
- Conserver les versions résolues avec package-lock.json.
- Exécuter des commandes standardisées dans l'équipe.

CARTE DE L'ÉCOSYSTÈME / APPRENDRE PAR LA PRATIQUE

Lire les fichiers d'un projet

Chaque fichier a un rôle distinct.

```
mon-projet/
├── package.json       # scripts et dépendances
├── package-lock.json  # versions résolues
├── src/               # code source
└── node_modules/      # paquets installés
```

> **RÉSULTAT ATTENDU**
>
> On versionne généralement package.json et package-lock.json, pas node_modules.

CARTE DE L'ÉCOSYSTÈME / APPRENDRE PAR LA PRATIQUE

Vite, React et Next.js

Ils interviennent à des niveaux différents.

- Vite : outil de build et serveur de développement.
- React : bibliothèque pour construire des interfaces.
- Next.js : framework basé sur React.

Vérification 01

Quel énoncé est correct ?

Quel outil permet d'exécuter du JavaScript côté serveur ?

A React

B Node.js

C CSS

Choisissez une réponse pour vérifier votre compréhension.

INSTALLATION & OUTILS / APPRENDRE PAR LA PRATIQUE

La boîte à outils minimale

Ne commencez pas par installer dix frameworks.

- Un éditeur : VS Code ou équivalent.
- Node.js et npm pour exécuter les scripts et gérer les paquets.
- Un terminal pour lancer les commandes.
- Un navigateur pour observer le comportement côté client.
- Git pour versionner le travail — utile dès le début.

INSTALLATION & OUTILS / APPRENDRE PAR LA PRATIQUE

Vérifier l'environnement

Avant de coder, vérifier les versions disponibles.

```
node --version
npm --version
```

> **RÉSULTAT ATTENDU**
>
> Les commandes doivent afficher des numéros de version.

INSTALLATION & OUTILS / APPRENDRE PAR LA PRATIQUE

Créer un dossier de travail

Un projet commence par un répertoire clair.

```
mkdir premiers-pas-js
cd premiers-pas-js
npm init -y
```

RÉSULTAT ATTENDU

npm crée un package.json initial.

INSTALLATION & OUTILS / APPRENDRE PAR LA PRATIQUE

Comment exécuter un fichier .js

Le terminal lance le runtime avec le fichier en argument.

```
// hello.js
co
nsole.log("Bonjour HashCode");

// terminal
node hello.js
```

INSTALLATION & OUTILS / APPRENDRE PAR LA PRATIQUE

Prédisez avant d'exécuter

Le réflexe d'un bon développeur est de prévoir le résultat.

```
const city = "Cotonou";
console.log("Bonjour " + city);
```

À VOUS

Écrivez mentalement la sortie exacte avant de lancer le programme.

Lire une erreur sans paniquer

Un message d’erreur est une information de diagnostic.

- Lire le type d’erreur avant de modifier le code.
- Repérer le fichier et la ligne concernés.
- Identifier la dernière ligne pertinente de la trace.
- Reproduire le problème avec le plus petit exemple possible.
- Changer une chose à la fois, puis retester.

HASHCODE / MASTERCLASS

Partie 01

Les briques de base : valeurs, décisions, fonctions et collections.

JAVASCRIPT — LES FONDAMENTAUX / APPRENDRE PAR LA PRATIQUE

Une variable est une liaison

Le nom permet de réutiliser une valeur dans le programme.

- const : la liaison ne sera pas réassignée.
- let : la liaison pourra recevoir une autre valeur.
- Éviter var dans les nouveaux exemples d'initiation.

```
const community = "HashCode";
let participants = 12;
```

const ou let ?

Choisir selon l'intention du programme.

A

const

- Valeur de liaison stable.
- Choix par défaut recommandé.
- Réduit les réassignations accidentelles.

B

let

- Valeur réassignée plus tard.
- Utile pour un compteur ou un état local.
- À utiliser seulement si nécessaire.

Vérification 02

Quelle déclaration correspond à un compteur qui augmente ?

Quelle option est la plus adaptée ?

A const count = 0; puis count = count + 1

B let count = 0; puis count = count + 1

C var count = 0; obligatoire

Choisissez une réponse pour vérifier votre compréhension.

JAVASCRIPT — LES FONDAMENTAUX / APPRENDRE PAR LA PRATIQUE

Les types primitifs à reconnaître

Le type influence les opérations possibles sur une valeur.

- string : texte — « HashCode ».
- number : entiers et décimaux — 42, 3.14.
- boolean : true ou false.
- null : absence volontaire de valeur.
- undefined : aucune valeur assignée.
- object : objets et tableaux.
- symbol et bigint : cas plus rares, à connaître de nom.

```
typeof "HashCode"; // "string"
typeof 42;         // "number"
typeof true;       // "boolean"
typeof undefined;  // "undefined"
typeof null;       // "object" (particularité historique)
```

Les conversions peuvent surprendre

JavaScript convertit parfois implicitement les valeurs.

```
console.log("5" + 2);
console.log("5" - 2);
console.log(Boolean(""));
```

RÉSULTAT ATTENDU

52\n3\nfalse

Comparer avec ===

L'égalité stricte évite la coercition de type de ==.

```
5 == "5"   // true
5 === "5"  // false
```

À RETENIR

En pratique, privilégiez === sauf raison explicite d'utiliser ==.

JAVASCRIPT — LES FONDAMENTAUX / APPRENDRE PAR LA PRATIQUE

Les conditions prennent une décision

if / else choisit un chemin selon une expression booléenne.

```
const score = 72;

if (score >= 50) {
  console.log("Validé");
} else {
  console.log("À retravailler");
}
```

RÉSULTAT ATTENDU

Validé

JAVASCRIPT — LES FONDAMENTAUX / APPRENDRE PAR LA PRATIQUE

Les boucles répètent une action

Utiliser une boucle quand une logique doit s'appliquer plusieurs fois.

```
for (let i = 1; i <= 3; i++) {
  console.log(`Étape ${i}`);
}
```

RÉSULTAT ATTENDU

Étape 1\nÉtape 2\nÉtape 3

JAVASCRIPT — LES FONDAMENTAUX / APPRENDRE PAR LA PRATIQUE

Une fonction nomme une action

Une fonction transforme des entrées en un résultat ou produit un effet.

```
function greet(name) {
  return `Bonjour ${name}`;
}

console.log(greet("Eurin"));
```

RÉSULTAT ATTENDU

```
Bonjour Eurin
```

JAVASCRIPT — LES FONDAMENTAUX / APPRENDRE PAR LA PRATIQUE

function ou fonction fléchée ?

Les deux formes sont utiles ; choisissez la plus lisible.

A

Fonction déclarée

- function sum(a, b) { return a + b; }
- Peut être appelée avant sa déclaration grâce au hoisting de la déclaration.

B

Fonction fléchée

- const sum = (a, b) => a + b;
- Courante pour les callbacks et fonctions courtes.
- Le comportement de this diffère.

JAVASCRIPT — LES FONDAMENTAUX / APPRENDRE PAR LA PRATIQUE

Un tableau garde une collection ordonnée

Les tableaux permettent d'itérer et de transformer des listes.

```
const prices = [1000, 2500, 1500];

console.log(prices.length);
console.log(prices[0]);
```

RÉSULTAT ATTENDU

3\n1000

JAVASCRIPT — LES FONDAMENTAUX / APPRENDRE PAR LA PRATIQUE

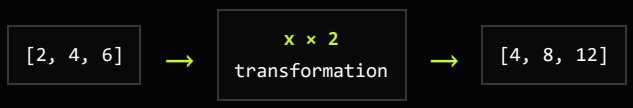
map transforme chaque élément

map renvoie un nouveau tableau de même longueur.

```
const prices = [1000, 2500, 1500];
const withTax = prices.map(price => price * 1.18);
```

MAP — TRANSFORMER CHAQUE ÉLÉMENT

ENTRÉE → SORTIE



Même nombre d'éléments ; chaque valeur de sortie correspond à une valeur transformée de l'entrée. Le tableau d'origine n'est pas modifié.

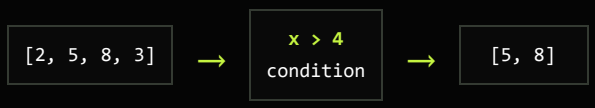
JAVASCRIPT — LES FONDAMENTAUX / APPRENDRE PAR LA PRATIQUE

filter sélectionne des éléments

filter garde uniquement les éléments dont le test renvoie true.

```
const scores = [35, 82, 49, 91];
const passed = scores.filter(score => score >= 50);
```

FILTER — SÉLECTIONNER ENTRÉE → SORTIE



Le prédicat renvoie true ou false pour chaque élément. Seuls les éléments qui satisfont la condition restent dans le nouveau tableau.

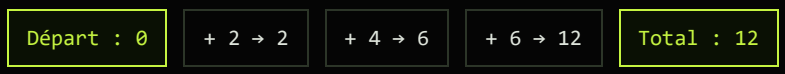
reduce accumule une valeur

reduce combine une collection en une valeur finale.

```
const prices = [1000, 2500, 1500];
const total = prices.reduce(
  (sum, price) => sum + price,
  0
);
```

REDUCE — ACCUMULER

ÉTAT DE L'ACCUMULATEUR



À chaque itération, reduce reçoit l'accumulateur courant et l'élément. Il renvoie une valeur finale, ici la somme.

JAVASCRIPT — LES FONDAMENTAUX / APPRENDRE PAR LA PRATIQUE

Un objet regroupe des propriétés

Les objets représentent une entité avec des champs nommés.

```
const task = {
  id: 1,
  title: "Apprendre JavaScript",
  completed: false,
};

console.log(task.title);
```

RÉSULTAT ATTENDU

```
Apprendre JavaScript
```

Des objets dans un tableau

C'est une structure fréquente dans les applications.

```
const tasks = [
  { id: 1, title: "Lire", completed: true },
  { id: 2, title: "Pratiquer", completed: false },
];

const todo = tasks.filter(task => !task.completed);
```

RÉSULTAT ATTENDU

```
[ { id: 2, title: "Pratiquer", completed: false } ]
```

JAVASCRIPT — LES FONDAMENTAUX / APPRENDRE PAR LA PRATIQUE

Exercice guidé : calculer un panier

Travail individuel — 3 minutes.

- Vous recevez [1200, 800, 2500].
- Calculez la somme sans écrire le résultat en dur.
- Affichez le total dans la console.
- Bonus : comptez le nombre d'articles.

À VOUS

Avant de coder, écrivez en une phrase ce que doit faire votre programme.

CRITÈRES DE RÉUSSITE

Total affiché = 4500 · Articles = 3 via length (bonus) · la somme est calculée par le code, jamais écrite en dur.

Correction : le panier

Une solution compacte avec reduce.

```
const prices = [1200, 800, 2500];
const total = prices.reduce((sum, price) => sum + price, 0);

console.log("Total :", total);
console.log("Articles :", prices.length);
```

RÉSULTAT ATTENDU

```
Total : 4500\nArticles : 3
```

Les valeurs peuvent être absentes

Les données réelles sont parfois incomplètes.

- Vérifier qu'une propriété existe avant de l'utiliser.
- Ne pas supposer que toutes les données reçues sont complètes.
- Utiliser des valeurs par défaut quand cela a du sens.

```
const user = { name: "Ada" };
console.log(user.email); // undefined
```

Vérification 03

Quel tableau est renvoyé ?

const nums = [1, 2, 3]; nums.map(n => n * 2)

A [1, 2, 3]

B [2, 4, 6]

C 6

Choisissez une réponse pour vérifier votre compréhension.

JavaScript : les repères essentiels

Si vous retenez cinq choses, retenez celles-ci.

- const par défaut ; let si réassignation nécessaire.
- === pour éviter les conversions implicites de ==.
- Une fonction isole une action ou un calcul.
- map transforme ; filter sélectionne ; reduce accumule.
- Un tableau d'objets est une structure de données très fréquente.

HASHCODE / MASTERCLASS

Partie 02

Ajouter des contrats au code pour détecter certaines erreurs plus tôt.

TYPESCRIPT — LES FONDAMENTAUX / APPRENDRE PAR LA PRATIQUE

Le problème que TypeScript aide à résoudre

Une erreur peut venir d'une valeur qui n'a pas la forme attendue.

```
function double(value) {
  return value * 2;
}

double("bonjour");
```

À RETENIR

La fonction attend-elle un nombre, une chaîne ou les deux ? Le code ne le dit pas.

RÉSULTAT ATTENDU

NaN — aucune erreur levée : le résultat est silencieusement faux.

Rendre le contrat explicite

TypeScript permet d’indiquer les types attendus.

A

JavaScript

- function double(value) {
- return value * 2;
- }
- Le contrat n’est pas déclaré.

B

TypeScript

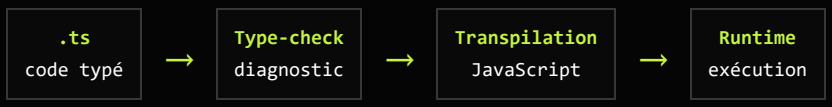
- function double(value: number): number {
- return value * 2;
- }
- L’éditeur et le compilateur peuvent signaler un argument incompatible.

Le typage est vérifié avant l'exécution

Les annotations TypeScript ne sont pas des contrôles automatiques à l'exécution.

- Le compilateur analyse les types statiquement.
- Le JavaScript émis ne conserve généralement pas les annotations.
- Une donnée reçue d'une API doit être validée à l'exécution.
- Le typage réduit certaines erreurs ; il ne supprime pas tous les bugs.

LE PARCOURS TYPESCRIPT — AVANT L'EXÉCUTION



TypeScript détecte des incohérences statiques ; il ne garantit pas que les données reçues à l'exécution sont valides et ne remplace pas les tests.

TYPESCRIPT — LES FONDAMENTAUX / APPRENDRE PAR LA PRATIQUE

Les annotations de base

Commencez avec les types les plus courants.

```
const community: string = "HashCode";
const members: number = 24;
const active: boolean = true;
```

RÉSULTAT ATTENDU

Aucune sortie (déclarations valides).

L’inférence évite le bruit

Le compilateur peut déduire le type depuis la valeur.

```
const city = "Cotonou"; // string
const count = 12;       // number

// L'annotation explicite n'est pas toujours utile.
```

À RETENIR

Règle pratique : annotez surtout les frontières, les paramètres et les structures importantes.

Typage des paramètres et du retour

Une fonction expose un contrat lisible.

```
function calculateTotal(
  price: number,
  quantity: number
): number {
  return price * quantity;
}
```

Vérification 04

Quel appel est incompatible avec la signature ?

function repeat(text: string, times: number): string

A repeat("Hi", 3)

B repeat("Code", 2)

C repeat(4, "deux")

Choisissez une réponse pour vérifier votre compréhension.

Les unions décrivent plusieurs cas

Une valeur peut appartenir à un ensemble de types autorisés.

```
type Status = "todo" | "doing" | "done";

const status: Status = "doing";
```

À RETENIR

Une union de littéraux limite les valeurs possibles et rend les états explicites.

Une interface décrit la forme d’un objet

Elle précise les propriétés et leurs types.

```
interface Task {
  id: number;
  title: string;
  completed: boolean;
}
```

Créer une valeur conforme

Le compilateur compare l'objet à l'interface.

```
const task: Task = {
  id: 1,
  title: "Comprendre TypeScript",
  completed: false,
};
```

RÉSULTAT ATTENDU

Objet conforme : aucune erreur de type.
Variante invalide (ex. completed: "non" ou title manquant) : erreur de vérification.

Propriété optionnelle ou obligatoire ?

Le point d'interrogation change le contrat.

- name doit être présent.
- bio peut être absent.
- Une propriété optionnelle peut nécessiter une vérification avant usage.

```
interface User {
  name: string;
  bio?: string;
}
```

Les tableaux peuvent être typés

Le type décrit les éléments autorisés dans la collection.

```
const scores: number[] = [12, 18, 15];
const tasks: Task[] = [];
```

Les types ne remplacent pas les tests

Un code peut respecter ses types et rester incorrect.

- Un calcul peut utiliser une mauvaise formule.
- Une règle métier peut être mal comprise.
- Une API peut renvoyer des données inattendues.
- Les tests vérifient le comportement ; les types vérifient des contraintes statiques.

Exercice : typer une fonction

Deux minutes pour écrire le contrat.

- Écrire une fonction formatTitle(title).
- Le paramètre doit être une chaîne.
- La fonction doit renvoyer une chaîne.
- Bonus : accepter une priorité numérique séparée.

À VOUS

Écrivez d'abord la signature, puis le corps.

CRITÈRES DE RÉUSSITE

Signature attendue : function formatTitle(title: string): string · le corps renvoie title.trim() · bonus : accepter un second paramètre numérique (priorité).

Correction : contrat explicite

La signature documente l'entrée et la sortie.

```
function formatTitle(title: string): string {
  return title.trim();
}
```

- title est une chaîne.
- trim renvoie une nouvelle chaîne.
- Le résultat est déclaré string.

TypeScript : les repères essentiels

Le typage est un outil de conception et de vérification.

- Les annotations décrivent les contrats.
- L'inférence évite d'annoter tout inutilement.
- Les interfaces décrivent la forme des objets.
- Les unions limitent les valeurs possibles.
- Les types ne remplacent ni validation d'entrée ni tests.

HASHCODE / MASTERCLASS

Partie 03

Assembler les concepts dans un mini-gestionnaire de tâches.

ATELIER — TASK MANAGER / APPRENDRE PAR LA PRATIQUE

Le problème à résoudre

Une petite équipe veut suivre ses tâches sans perdre le fil.

01

AJOUTER

Créer une tâche avec un titre valide.

02

AFFICHER

Lister les tâches et leur état.

03

TERMINER

Marquer une tâche comme terminée.

04

PROTÉGER

Refuser un titre vide et les identifiants inconnus.

05

SUPPRIMER

Retirer une tâche par son identifiant.

Le modèle de données

Définir la forme d’une tâche avant les fonctions.

PONT

Avant, une tâche portait completed: boolean (slides 39-40) ; ici, ce booléen devient un état explicite status: "todo" | "done".

```
type TaskStatus = "todo" | "done";

interface Task {
  id: number;
  title: string;
  status: TaskStatus;
}
```

ATELIER — TASK MANAGER / APPRENDRE PAR LA PRATIQUE

Séparer les responsabilités

Chaque fonction doit avoir une responsabilité facile à expliquer.

01

addTask

Valider et ajouter

02

listTasks

Afficher la collection

03

completeTask

Modifier l'état

04

removeTask

Retirer un élément

SÉPARER LES RESPONSABILITÉS — TASK MANAGER



Une responsabilité claire facilite les tests, le débogage et les évolutions. Pour cet atelier en console, l'interface peut simplement être la sortie du terminal.

ATELIER — TASK MANAGER / APPRENDRE PAR LA PRATIQUE

Le point de départ

Un tableau typé conserve les tâches pendant l'exécution.

```
const tasks: Task[] = [];
let nextId = 1;
```

- tasks est une collection de Task.
- nextId produit un identifiant simple pour la démonstration.
- Les données ne sont pas persistées après la fermeture du programme.

ATELIER — TASK MANAGER / APPRENDRE PAR LA PRATIQUE

Ajouter une tâche

Valider l'entrée avant de modifier l'état.

```
function addTask(title: string): Task {
  const cleanTitle = title.trim();
  if (!cleanTitle) {
    throw new Error("Le titre est obligatoire");
  }
  const task: Task = {
    id: nextId++,
    title: cleanTitle,
    status: "todo",
  };
  tasks.push(task);
  return task;
}
```

ATELIER — TASK MANAGER / APPRENDRE PAR LA PRATIQUE

Afficher les tâches

Séparer la représentation des données de leur affichage.

```
function listTasks(): void {
  if (tasks.length === 0) {
    console.log("Aucune tâche.");
    return;
  }
  for (const task of tasks) {
    const mark = task.status === "done" ? "✓" : "○";
    console.log(`${mark} ${task.id}. ${task.title}`);
  }
}
```

ATELIER — TASK MANAGER / APPRENDRE PAR LA PRATIQUE

Terminer une tâche

Rechercher par identifiant, puis modifier son état.

```
function completeTask(id: number): boolean {
  const task = tasks.find(item => item.id === id);
  if (!task) return false;
  task.status = "done";
  return true;
}
```

ATELIER — TASK MANAGER / APPRENDRE PAR LA PRATIQUE

Prédisez le résultat

Avant de lancer le programme, suivez l'état étape par étape.

```
addTask("Lire la documentation");
addTask("Pratiquer les types");
completeTask(1);
listTasks();
```

À VOUS

Quelle tâche sera marquée comme terminée ? Quel symbole affichera la seconde ?

ATELIER — TASK MANAGER / APPRENDRE PAR LA PRATIQUE

À vous : ajoutez une règle

Travail en binôme ou individuel — 5 minutes.

- Empêcher les doublons : refuser un titre déjà présent.
- Afficher un message si l'identifiant est inconnu.
- Ajouter une fonction removeTask(id).
- Vérifier le résultat après chaque changement.

À VOUS

Testez au moins un cas normal et un cas limite.

ATELIER — TASK MANAGER / APPRENDRE PAR LA PRATIQUE

Défi : supprimer une tâche

Une recherche renvoie un index ou -1.

```
function removeTask(id: number): boolean {
  const index = tasks.findIndex(task => task.id === id);
  if (index === -1) return false;
  tasks.splice(index, 1);
  return true;
}
```

- findIndex retourne -1 si rien ne correspond.
- splice modifie le tableau en place.
- Le booléen permet à l'appelant de réagir.

ATELIER — TASK MANAGER / APPRENDRE PAR LA PRATIQUE

Vérifier le type et le comportement

Deux contrôles complémentaires.

Mini-plan de test

Validez le gestionnaire avec trois scénarios.

- Ajouter « Préparer la masterclass » : la tâche apparaît.
- Terminer l'identifiant 1 : son état devient done.
- Terminer l'identifiant 999 : le programme signale l'échec sans planter.

À VOUS

Notez le résultat attendu avant d'exécuter chaque scénario.

```
if (!completeTask(999)) {
  console.log("Identifiant inconnu : 999");
}
```

Ce que nous venons de construire

Un mini-projet utile pour relier plusieurs concepts.

- Une interface décrit les données.
- Les fonctions séparent les responsabilités.
- Les tableaux stockent et parcourent les tâches.
- La validation protège les entrées.
- Les tests vérifient les comportements.

ANCRAGE & SUITE / APPRENDRE PAR LA PRATIQUE

Vérification 05

Quel mécanisme vérifie les annotations TypeScript ?

Quand détecte-t-on généralement une incompatibilité de type ?

A Pendant le contrôle statique / compilation

B Uniquement après que l'utilisateur clique

C Grâce au CSS

Choisissez une réponse pour vérifier votre compréhension.

ANCRAGE & SUITE / APPRENDRE PAR LA PRATIQUE

Vérification 06

Quelle méthode choisit les éléments qui respectent une condition ?

Quel outil de tableau utiliser pour ne garder que les tâches non terminées ?

- A map
- B filter
- C reduce

Choisissez une réponse pour vérifier votre compréhension.

TOTAL SESSION

0 / 6 · 0 pt

ANCRAGE & SUITE / APPRENDRE PAR LA PRATIQUE

Le réflexe à garder

Avant d’écrire du code, posez trois questions.

- Quelles sont les données d’entrée ?
- Quel résultat ou effet est attendu ?
- Comment vais-je vérifier que ça fonctionne ?

À VOUS

La qualité du raisonnement compte davantage que la vitesse de frappe.

ANCRAGE & SUITE / APPRENDRE PAR LA PRATIQUE

Votre prochain challenge HashCode

Choisissez un seul niveau — 15 minutes — puis expliquez votre solution.

01

NIVEAU 1

Ajouter une priorité basse / normale / haute.

02

NIVEAU 2

Filtrer avec filter(status === "done") et afficher le nombre restant.

03

NIVEAU 3

Écrire des tests et séparer modèle / affichage.

ANCRAGE & SUITE / APPRENDRE PAR LA PRATIQUE

Plan de pratique sur 7 jours

Une courte pratique quotidienne vaut mieux qu'une longue session isolée.

- Jour 1 : variables, types et opérateurs.
- Jour 2 : conditions et boucles.
- Jour 3 : fonctions et tableaux.
- Jour 4 : objets et méthodes de tableau.
- Jour 5 : types et interfaces TypeScript.
- Jour 6 : finir Task Manager.
- Jour 7 : expliquer son code et noter les questions.

ANCRAGE & SUITE / APPRENDRE PAR LA PRATIQUE

Les cinq idées à emporter

Une base solide se construit avec des distinctions claires.

- JavaScript est le langage exécuté ; TypeScript ajoute un contrôle statique.
- Le navigateur et Node.js sont des environnements différents.
- Les fonctions donnent un nom aux actions et calculs.
- map transforme, filter sélectionne, reduce accumule.
- Les types, les tests et la validation se complètent.

ANCRAGE & SUITE / APPRENDRE PAR LA PRATIQUE

Comprendre. Construire. Partager.

La suite commence avec un petit projet que vous pouvez expliquer.

À FAIRE APRÈS LA SÉANCE

Terminez un niveau du challenge, prenez une photo ou une capture de votre code, puis partagez-la dans la communauté HashCode avec une phrase d'explication. Dépôt du support : github.com/digitaleflex/hashcode-masterclass